



RESEARCH ARTICLE

TRIDENT: GPU-Accelerated EKF Positioning for Downhole Traction Robots

First Author¹✉, Second Author¹

1. Department, University, City Postal Code, China

Received month dd, yyyy; accepted month dd, yyyy

E-mail: corresponding.author@example.com.

© Higher Education Press 2026

Abstract

Extended Kalman filter (EKF)-based multi-sensor fusion underpins positioning algorithms for robots that operate without reliable external positioning. In simulations of a downhole traction robot in a cased horizontal well, the estimator processes a long sensor sequence through tightly dependent SINS propagation and EKF correction steps. Its small matrices and fine-grained operators form a latency-bound GPU workload: a direct tensor implementation expands each filter step into hundreds of dependent kernels and leaves most streaming multiprocessors idle. We present TRIDENT, a three-axis GPU acceleration framework for this workload. First, a Triton whole-scan kernel fuses the complete recursive trajectory, keeps the recurrent navigation and filter state on chip, and replaces the general innovation solve with an analytical small-matrix inverse. Second, component-aware specialization separates the precision of sensor input, recurrent computation, and dot products to control long-horizon numerical error while reducing instruction and storage cost. Third, independent simulation trajectories are mapped to CUDA blocks to expose batch-level parallelism without changing the temporal order within each filter. On a Hopper-class GPU, whole-scan fusion in fp64 delivers an 8.8× speedup over the CPU reference, and the fused fp32 implementation reaches approximately 2.43×10^5 filter steps per second—a 130× single-trajectory speedup with sub-millimeter deviation from the fp64 trajectory. Multi-trajectory execution scales aggregate throughput to 4.63×10^7 filter steps per second before register-limited saturation. These results show how execution granularity, numerical representation, and outer-loop parallelism can be coordinated for long-horizon recursive state-estimation simulation.

1 Introduction

Embodied artificial intelligence (AI) is extending autonomous machines from structured, instrumented spaces into physical environments with uncertain contacts, limited visibility, and unreliable external positioning [1]. A robot must maintain an estimate of its pose, velocity, and internal state while it perceives the environment, plans motion, and applies control. Deep-space vehicles, subterranean platforms, legged robots, and autonomous vehicles use different sensors and motion models, yet their local state-estimation loops repeatedly propagate inertial information and correct it with intermittent observations through recursive Bayesian estimation [2, 3]. The quality and timeliness of this loop determine how reliably the rest of the perception–action system can operate.

This requirement is especially visible in embodied-AI systems [4]. Humanoids and quadrupeds combine inertial, kinematic, contact, and visual measurements at high update rates; a local extended Kalman filter (EKF) or error-state filter often provides the deterministic propagation layer even when a separate mapping system supplies global consistency. Consistent fusion of leg kinematics and inertial measurements has been demonstrated for legged robots [5], while the DARPA Subterranean Challenge shows how localization, mapping, and navigation must remain tightly coupled when robots operate in underground

environments [6]. The same systems increasingly depend on large simulation campaigns that vary contact conditions, sensor noise, terrain, and control parameters. GPU-native simulators such as Isaac Gym execute thousands of robot environments concurrently [7], raising the throughput requirement of every repeated state-estimation loop in the simulation pipeline.

Downhole traction robots provide a concrete and demanding instance of this broader embodied-autonomy problem. These robots actively convey coiled tubing, toolstrings, and attached tools through highly deviated and horizontal wells, where gravity, friction, and long constrained passages make passive delivery unreliable. Recent field deployments have demonstrated their use in horizontal-well intervention, logging, perforation, and zonal-isolation operations [8, 9]. As illustrated in Figure 1, a surface unit deploys the toolstring through the wellbore while the tractor provides active conveyance along the horizontal section. In the cased horizontal well considered in this study, high temperature and pressure, corrosion, deposits, local deformation, and obstructions can perturb wheel–wall contact and induce slip or short-lived odometry disturbances. Reliable operation therefore requires continuous fusion of inertial and traction measurements to estimate the robot state and suppress accumulated drift. Before deployment, large simulation campaigns are needed to evaluate sensor noise, slip

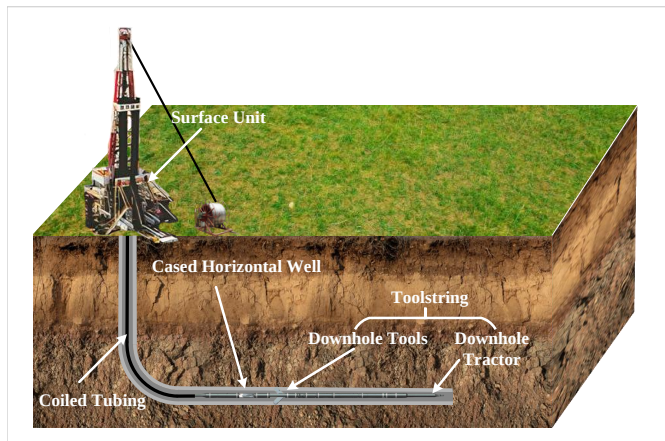


Fig. 1 Schematic of a downhole traction robot conveying a toolstring into the horizontal section of an oil or gas well.

events, obstacle-induced disturbances, and filter parameters over long trajectories, making downhole traction robot positioning not only a state-estimation problem but also a computationally intensive simulation workload.

The resulting estimator has an execution pattern that differs from the large, throughput-oriented matrix workloads commonly used to showcase GPUs. Each update is a small numerical operation embedded in a long dependency chain, and a direct tensor implementation fragments the computation into many fine-grained device operations. Repeated launches, synchronization, and movement of recurrent state can dominate arithmetic, leaving much of the accelerator underutilized and causing a straightforward GPU port to underperform a CPU implementation. These characteristics motivate an implementation study centered on launch overhead, on-chip state locality, numerical precision, and independent trajectories available at the simulation level.

We develop TRIDENT, a GPU acceleration framework for this workload. Operator fusion removes fragmented execution and intermediate-state traffic; component-aware precision matches numerical requirements to GPU instruction throughput; and multi-trajectory parallelism fills otherwise idle SMs while preserving the temporal dependency within each trajectory. The main contributions are as follows:

- We diagnose the execution behavior of SINS/EKF simulation and develop a progressive fusion path culminating in a single Triton megakernel that keeps the recurrent filter state on chip and executes a complete trajectory with minimal device dispatch.
- We establish a component-aware precision strategy for recursive filtering and evaluate the performance and long-horizon accuracy of fp64, fp32, reduced-precision input, bfloat16, and TensorFloat-32 paths, including failure cases caused by error accumulation.
- We map independent trajectories to CUDA blocks and derive a scaling model that relates batch throughput to accelerator occupancy and register pressure, enabling high-throughput Monte Carlo and parameter-sweep simulation.

The remaining sections of this paper are organized as follows. Section 2 introduces the positioning model and its computational

structure. Section 3 presents the profiling evidence and optimization requirements. Section 4 describes the TRIDENT framework. Section 5 evaluates performance, accuracy, and scalability. Section 6 discusses related work and Section 7 concludes the paper.

■ 2 Background

■ 2.1 Downhole Traction Robot Positioning

Downhole traction robots actively convey coiled tubing, toolstrings, and payloads through highly deviated and horizontal wells. The deployment configuration in Figure 1 shows the tractor moving with the toolstring while its drive elements maintain contact with the well wall. This study considers operation inside a cased horizontal well, where the available space is narrow and absolute positioning signals are generally unavailable. High temperature, pressure, corrosion, well-wall deformation, deposits, and local obstacles can change the contact conditions between the robot and the casing. These effects can introduce wheel slip and short-duration disturbances while the robot performs traction, logging, inspection, or other downhole operations. A positioning system must therefore maintain a local state estimate over a long constrained motion and remain useful when an individual measurement becomes unreliable.

The simulated platform uses a three-axis inertial measurement unit (IMU) and two wheel odometers. Gyroscope and accelerometer measurements support high-rate propagation of attitude, velocity, and position, but their biases cause the propagated solution to drift over time. The odometers provide an independent estimate of traveled distance and can correct this drift, while disagreement between the two odometers or between odometry and inertial propagation provides a signal for detecting local disturbances. The fusion system uses these complementary measurements in a local navigation frame whose origin and initial attitude are set during initialization.

The casing geometry supplies additional information about feasible robot motion. During nominal traction, displacement is dominated by the direction of the well axis, whereas sustained lateral or radial velocity is inconsistent with the confined workspace. These motion constraints improve local observability when absolute position is unavailable. They cannot, however, be treated as uniformly reliable: wheel slip, asymmetric traction, and obstacle contact can produce transient measurements that violate the nominal motion pattern. The estimator must therefore combine continuous inertial propagation with measurement selection, using the available odometry information without allowing a short-lived contact disturbance to corrupt the full trajectory.

■ 2.2 SINS/EKF Fusion Model

The simulation follows a strapdown inertial navigation system (SINS) with an error-state extended Kalman filter. At sample k , the SINS first updates the quaternion attitude from the angular-rate measurement, converts the measured specific force from the body frame to the navigation frame, subtracts gravity, and integrates velocity and position. Quaternion normalization is applied after each update to keep the attitude representation bounded.

The SINS state is maintained as a nominal navigation solution, while the EKF represents only small deviations from that solution. This

error-state formulation separates the nonlinear integration of attitude and motion from the locally linear uncertainty update. It also supports closed-loop correction: after the EKF estimates the navigation errors, the nominal position, velocity, and attitude are corrected and the corresponding error coordinates are reset. Sensor biases remain part of the estimated state because even small bias errors can accumulate during a long period without an external position reference.

The EKF estimates the error of the nominal SINS state. Its 15-dimensional error vector is

$$\mathbf{x}_k = [\delta \mathbf{p}_k^T, \delta \mathbf{v}_k^T, \boldsymbol{\phi}_k^T, \boldsymbol{\varepsilon}_k^T, \nabla_k^T]^T \in \mathbb{R}^{15}, \quad (1)$$

where the five three-dimensional components represent position error, velocity error, attitude error, gyroscope bias, and accelerometer bias, respectively.

The linearized transition matrix is constructed from the current attitude, specific force, angular rate, and sampling interval. The covariance and error state are then propagated and corrected through the standard EKF prediction and measurement update:

$$\mathbf{x}_k^- = \mathbf{F}_k \mathbf{x}_{k-1}, \quad (2)$$

$$\mathbf{P}_k^- = \mathbf{F}_k \mathbf{P}_{k-1} \mathbf{F}_k^T + \mathbf{Q}_k, \quad (3)$$

$$\mathbf{K}_k = \mathbf{P}_k^- \mathbf{H}_k^T (\mathbf{H}_k \mathbf{P}_k^- \mathbf{H}_k^T + \mathbf{R}_k)^{-1}, \quad (4)$$

$$\mathbf{x}_k = \mathbf{x}_k^- + \mathbf{K}_k (\mathbf{z}_k - \mathbf{H}_k \mathbf{x}_k^-), \quad (5)$$

$$\mathbf{P}_k = (\mathbf{I} - \mathbf{K}_k \mathbf{H}_k) \mathbf{P}_k^-. \quad (6)$$

Here, $\mathbf{F}_k$ is the linearized state-transition matrix, $\mathbf{P}_k$ is the error covariance, and $\mathbf{Q}_k$ models process uncertainty. The observation matrix $\mathbf{H}_k$ maps the error state into the currently available measurement space, while $\mathbf{R}_k$ represents measurement uncertainty. The matrix inverted in the gain calculation is the innovation covariance. Although its dimension is small in this application, its value changes with the propagated covariance and the accepted measurement set at every sample.

The measurement model uses the odometry displacement together with lateral and radial velocity constraints when the odometry consistency test accepts both channels. If a channel is judged to be affected by a local obstacle, the update uses only the remaining reliable constraints. The estimated position, velocity, and attitude errors are fed back to the nominal SINS state, and the error state is reset before the next sample. This feedback loop makes every update depend on the state and covariance produced at the previous sample.

2.3 Computational Characteristics

The overall computation is summarized in Figure 2. The resulting workload is a long sequential scan of sensor samples. Each step combines quaternion and vector operations with covariance propagation, a small innovation calculation, and several products involving matrices whose logical dimensions are no larger than 15×15 . The arithmetic intensity of an individual step is consequently modest, while the recurrent state and covariance must remain available for the next step. Temporal parallelism within one trajectory is therefore limited by the estimator semantics.

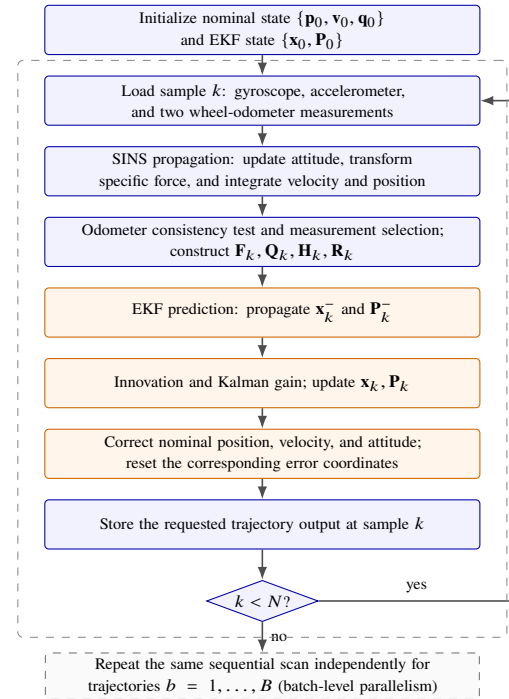


Fig. 2 Overall computational flow of the SINS/EKF positioning simulation. Operations within one trajectory follow a loop-carried dependency, whereas different trajectories have independent recurrent states and can be evaluated in parallel.

The covariance alone contains 225 logical elements, yet this state is still too small to provide the spatial parallelism of a large dense matrix operation. Moreover, the matrices used at a given sample are assembled from the current navigation state and measurement-validity decision rather than reused unchanged across the sequence. The workload therefore alternates small matrix products with quaternion normalization, coordinate transforms, scalar tests, and feedback operations. Its defining feature is not the cost of any individual operator, but the repeated execution of this heterogeneous operator chain under a loop-carried dependency.

The simulation nevertheless exposes a second form of parallelism. Trajectories generated with different noise realizations, obstacle configurations, or filter parameters do not share state and can be evaluated independently. Increasing the trajectory length adds sequential work, whereas increasing the number of trajectories adds independent work. The workload thus combines a sequential dependency along each trajectory with batch-level independence across trajectories. This combination defines the execution target for the optimization methods developed in the remaining sections of the paper. For fixed-shape kernel implementations, the logical filter matrices may be stored in padded 16×16 tiles without changing the 15-state model.

3 Motivation

3.1 Fine-Grained EKF Execution Is Latency-Bound

The CPU fp64 implementation processes approximately 1,866 filter steps per second and serves as the numerical reference; its representative characteristics are summarized in Table 1. Moving the same tensor-level

computation directly to the GPU does not produce a corresponding speedup. Each EKF step contains a dependent chain of tensor construction, element-wise arithmetic, trigonometric functions, and small matrix operations. The eager implementation expands this modest amount of arithmetic into roughly 531 CUDA kernel launches per step. As a result, dispatch, synchronization, and recurrent-state transfers occupy a substantial part of the execution time.

Device measurements confirm that the baseline is latency-bound. A single trajectory activates only a small portion of the GPU, and its measured SM throughput is approximately 0.27%. The launch cost of a general-purpose small-matrix kernel can approach or exceed the cost of its arithmetic. CUDA Graph replay reduces repeated host dispatch, but the captured graph still executes the fine-grained dependent kernels and transfers state between them. These observations motivate a larger execution scope that can preserve recurrent data across filter steps and eliminate fragmented device execution.

Table 1 Representative characteristics of the baseline positioning workload.

Characteristic	Value
Trajectory length	166,667 steps
EKF state dimension	15
Maximum logical matrix size	15×15
Naive kernels per step	approximately 531
CPU fp64 throughput	1,866 steps/s
Naive GPU SM throughput	approximately 0.27%

3.2 Precision Choices Have Unequal Numerical and Hardware Effects

Uniformly changing the precision of the estimator overlooks the different roles of its components. Sensor samples are noisy external inputs that are consumed once, whereas position, velocity, attitude, error state, and covariance are carried through the complete trajectory. Rounding in these recurrent quantities can accumulate through repeated propagation and feedback. Accuracy measured for an isolated filter step is therefore insufficient to determine whether a reduced-precision configuration is acceptable over a long simulation.

Lower nominal precision also does not guarantee lower execution time. TensorFloat-32 and Tensor Core paths may require conversion, padding, and scheduling work that is difficult to amortize for the small matrices in this estimator. Storage precision, recurrent arithmetic, and matrix-multiplication mode consequently present different accuracy and performance trade-offs. This behavior motivates component-level precision choices evaluated with both hardware measurements and complete-trajectory numerical error. Figure 3 combines the two views for the complete trajectory: throughput (bars) and the per-axis deviation from the all-fp32 reference (lines) share one panel, so accuracy and performance can be read together for each configuration.

We additionally isolate one precision decision at a time over the complete 166,667-step trajectory, while retaining fp32/IEEE arithmetic for all remaining components. As summarized in Table 2, fp16 sensor input causes only limited additional error, whereas bfloat16

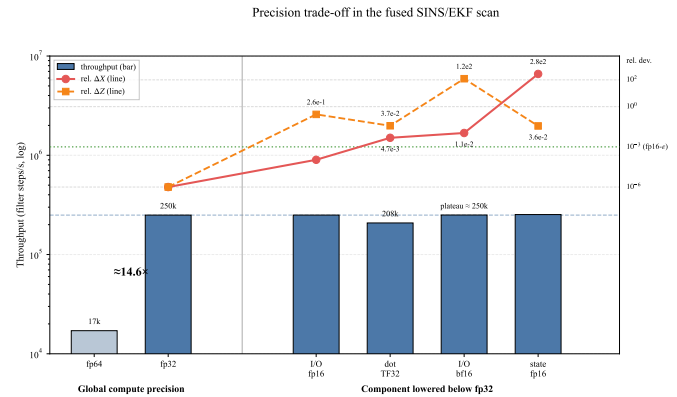


Fig. 3 Precision trade-off in the fused SINS/EKF scan over the full 166,667-step trajectory. Bars (lower log axis) report end-to-end throughput, and lines (upper band, log axis) report the relative X/Z positioning deviation against the all-fp32 reference. The 10^{-3} line marks an fp16- ϵ accuracy grade: bfloat16 sensor input and fp16 recurrent storage cross it and are unsafe for the long-horizon filter, whereas fp16 sensor input and TF32 matrix products remain near it without reducing throughput.

input substantially amplifies the vertical error. TF32 matrix products introduce a measurable X-axis deviation, and storing the recurrent navigation state in fp16 causes the long-horizon filter to diverge. These outcomes distinguish once-consumed sensor data from values repeatedly propagated through the recursion and motivate the component-aware scheme in Section 4.

Table 2 Component-isolation precision study over the full 166,667-step trajectory. Each row lowers one component while the remaining components use fp32/IEEE arithmetic; values report the RMSE increase relative to the all-fp32 reference.

Component	Lowered mode	ΔX (mm)	ΔZ (mm)
Sensor input	fp16	0.11	2.19
Matrix products	TF32	4.82	0.31
Sensor input	bfloat16	11.01	1005.84
Recurrent state	fp16 storage	280753.72	-0.30

3.3 Independent Trajectories Supply Device-Level Parallelism

The state and covariance at time k depend on the result at time $k-1$, which limits temporal parallelism within one trajectory. Increasing the tile size or assigning more threads to an individual small matrix product cannot create work along this dependency chain. Kernel fusion removes dispatch and state-traffic overhead, but it does not change this limit: a single fused trajectory launches one 128-thread block, or four warps. On the 78-SM Hopper device, this block occupies only one SM at 6.25% warp occupancy, leaving the device almost entirely idle, as summarized in Table 3. Reducing single-trajectory latency alone therefore cannot fill the GPU.

Simulation campaigns naturally provide an outer parallel dimension. Different noise realizations, obstacle configurations, and filter parameters produce independent trajectories with no shared recurrent state. Mapping these trajectories to separate blocks preserves the original update order within every filter while increasing aggregate

device utilization. Scaling eventually becomes limited by the number of SMs and by register-constrained block residency. Together, the three observations above lead to whole-scan kernel fusion, component-aware precision, and multi-trajectory parallelism, which are developed in the next section.

Table 3 Measured utilization for one fused fp32 trajectory on a 78-SM Hopper GPU. The kernel uses one 128-thread block per trajectory.

Characteristic	Value
Grid / block size	(1, 1, 1) / 128 threads (4 warps)
SMs hosting the block	1 of 78
Achieved occupancy	6.25% (4 / 64 warps)
Device SM throughput	approximately 0.27%

4 Methodology

4.1 Overview

TRIDENT models each positioning trajectory as a recursive scan over N sensor samples. At step k , the scan reads the current IMU and odometer measurements, propagates the nominal SINS state, performs EKF prediction and correction, feeds the estimated error back to the nominal state, and carries the updated navigation state and covariance to step $k + 1$. A batch of B simulations adds an outer trajectory dimension to the input and output tensors; the recurrent state remains private to each trajectory.

Figure 4 summarizes three complementary optimizations. First, whole-scan fusion encloses the complete time loop in one kernel, retains recurrent state on chip, and specializes the small-matrix operations on the critical path. Second, component-aware precision specialization assigns representations according to numerical role and selects a compile-time precision plan under a trajectory-level error constraint. Third, multi-trajectory mapping assigns independent scans to separate CUDA blocks, exposing device-wide parallelism subject to register-residency limits. The three axes target different bottlenecks—dispatch and data movement, arithmetic and storage cost, and insufficient device-level parallelism—and can therefore be composed.

These transformations preserve the SINS/EKF equations and the order of propagation, measurement update, and feedback within every trajectory. We use the CPU fp64 implementation as the semantic reference, compare complete output trajectories rather than isolated steps, and repeat identical trajectories across blocks to detect accidental state sharing. Section 5 reports the corresponding performance and accuracy results.

4.2 From Fragmented Operators to a Fused Recursive Scan

Figure 5(a) contrasts the eager and whole-scan fusion boundaries. In the eager implementation, one filter step expands into approximately 531 dependent kernels. Recurrent values therefore cross kernel boundaries and return to HBM hundreds of times per step, even though the next operation immediately consumes them.

We first regularize the step for static compilation. Scalar tensor construction, host-visible scalar access, and dynamic concatenation

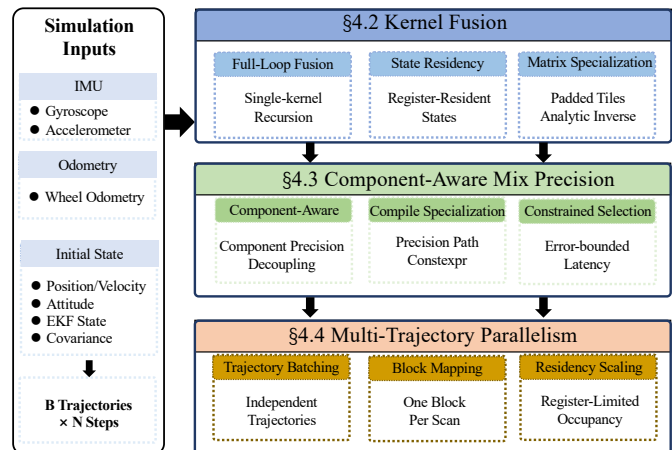


Fig. 4 Overview of TRIDENT. Kernel fusion retains recurrent state on chip, component-aware mixed precision selects numerically safe precision paths, and block-level trajectory mapping exposes device-wide parallelism.

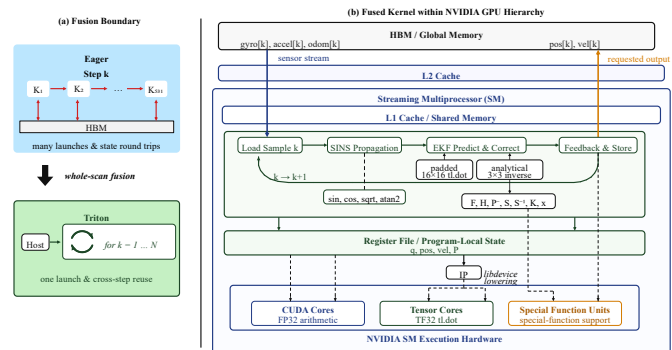


Fig. 5 Whole-scan fusion and its GPU mapping. (a) Fusion replaces hundreds of eager kernels per step with one Triton launch for the full trajectory. (b) The fused program streams sensor samples and requested outputs through HBM while retaining recurrent state in registers and mapping matrix and special-function operations to the corresponding SM execution units.

are replaced with device-side expressions. The two possible observation shapes use one fixed three-dimensional representation; when the odometry consistency test rejects a displacement observation, a large variance suppresses the corresponding Kalman gain without changing the matrix shape. CUDA Graph replay then removes repeated host dispatch, but the captured graph still contains the fine-grained kernels and their recurrent-state transfers. Whole-scan fusion removes this remaining boundary by placing all N steps inside one Triton program. A single host launch enters the scan, and the loop preserves the original update order while reusing state across iterations.

Figure 5(b) shows the resulting data path. At iteration k , the program streams only the current gyroscope, accelerometer, and odometer samples through L2 and the SM cache hierarchy. It performs SINS propagation, EKF prediction and correction, and feedback before advancing to $k + 1$. Only the requested position and velocity outputs are written to global memory. The quaternion, position, velocity, error state, and covariance remain program-local, so the loop-carried dependency passes through the register file instead of intermediate global-memory

tensors.

Two in-kernel transformations make the loop static and self-contained. First, masked 16×16 tiles store the logical 15-state filter matrices, providing a common layout for transition, covariance, observation, and gain products. The compile-time dot-product mode lowers these products to either CUDA-core IEEE arithmetic or Tensor Core TF32 operations. Second, the kernel computes the inverse of the 3×3 innovation covariance from its adjugate and determinant, avoiding a general solver and its synchronization point. SINS transcendental functions, including sine, cosine, square root, and arctangent, follow the device special-function path. The mapping in Figure 5(b) thus combines one fusion boundary, register-resident recurrence, and hardware-specific micro-operations to reduce dispatch, synchronization, and HBM traffic on the latency-bound critical path.

■ 4.3 Component-Aware Precision Specialization

Uniform precision ignores the different numerical roles within the scan. Sensor samples are noisy, once-consumed inputs, whereas position, velocity, attitude, error state, and covariance recur across the complete trajectory and can accumulate rounding error. Small matrix products introduce a third choice because TensorFloat-32 (TF32) changes the multiplication path without changing the surrounding storage type. We therefore treat input storage, recurrent arithmetic, and dot-product mode as independent dimensions.

The fused kernel exposes these dimensions as three compile-time parameters. DT_{IO} specifies the stored type of IMU and odometer samples, which are converted on load to the internal arithmetic type DT . The latter governs quaternion normalization, SINS integration, EKF propagation, covariance update, and feedback. IP selects the Triton dot-product mode, distinguishing IEEE and TF32 execution. Because all three parameters are compile-time constants, each combination produces a branch-free specialized kernel. We define a precision plan as $P = (DT_{IO}, DT, IP)$ over the candidate sets in Table 4.

Table 4 Precision dimensions exposed by the fused implementation.

Component	Candidates	Treatment
Sensor input	fp16, bf16, fp32	cast to DT
Recursive computation	fp32, fp64	retain across steps
Small matrix products	IEEE, TF32	IP specialization
Validation reference	fp64	CPU trajectory

We screen each plan over the complete trajectory rather than an isolated filter step. Let $\mathbf{p}_{0:N}^P$ be the position trajectory produced by plan P , and let $\mathbf{p}_{0:N}^{\text{ref}}$ be the CPU fp64 trajectory. The precision-induced deviation is $\delta_P = \text{RMSE}(\mathbf{p}_{0:N}^P, \mathbf{p}_{0:N}^{\text{ref}})$. Evaluating all N steps exposes errors that emerge only after repeated integration or covariance propagation. We separately define the irreducible model error as $\varepsilon_{\text{model}} = \text{RMSE}(\mathbf{p}_{0:N}^{\text{ref}}, \mathbf{p}_{0:N}^{\text{truth}})$. For a dimensionless tolerance α , plan P is acceptable when

$$\delta_P \leq \alpha \varepsilon_{\text{model}}, \quad (7)$$

so that numerical error introduced by the precision plan remains small relative to the model error already present in the fp64 reference. This

criterion separates numerical acceptance from hardware behavior: DT_{IO} changes input storage and bandwidth, DT changes recurrent arithmetic and state representation, and IP changes dot-product execution. Among the acceptable plans, we select the one with the lowest critical-path latency τ_P , breaking ties by the smaller register footprint r_P . We also retain the nondominated precision–latency set for analysis in Section 5.4. Algorithm 1 summarizes the procedure.

Algorithm 1 Component-Aware Precision Plan Selection

Require: candidate sets $\Pi_{IO}, \Pi_R, \Pi_{\text{dot}}$; sensor stream $\mathbf{Z}_{0:N-1}$; fp64 position trajectory $\mathbf{p}_{0:N}^{\text{ref}}$; model error $\varepsilon_{\text{model}}$; acceptance ratio α
Ensure: selected plan $P^* = (DT_{IO}^*, DT^*, IP^*)$ and Pareto front $\mathcal{F}$

- 1: $C \leftarrow \Pi_{IO} \times \Pi_R \times \Pi_{\text{dot}}$ $\triangleright$ enumerate component-wise candidates
- 2: $\mathcal{R} \leftarrow \emptyset$ $\triangleright$ error, latency, and register footprint
- 3: **for all** $P = (DT_{IO}, DT, IP) \in C$ **do**
- 4: $K_P \leftarrow \text{SPECIALIZE}(DT_{IO}, DT, IP)$ $\triangleright$ compile-time constants; no runtime branch
- 5: $(\mathbf{p}_{0:N}^P, \tau_P, r_P) \leftarrow \text{RUNTRAJECTORY}(K_P, \mathbf{Z}_{0:N-1})$ $\triangleright$ full N -step scan
- 6: $\delta_P \leftarrow \text{RMSE}(\mathbf{p}_{0:N}^P, \mathbf{p}_{0:N}^{\text{ref}})$ $\triangleright$ trajectory-level deviation
- 7: **if** $\delta_P \leq \alpha \varepsilon_{\text{model}}$ **then** $\triangleright$ accept by Equation (7)
- 8: $\mathcal{R} \leftarrow \mathcal{R} \cup \{(P, \delta_P, \tau_P, r_P)\}$
- 9: **end if**
- 10: **end for**
- 11: $\mathcal{F} \leftarrow \text{PARETOFRONT}(\mathcal{R}; \text{minimize } \delta_P, \text{minimize } \tau_P)$
- 12: $P^* \leftarrow \arg \min_{(P, \delta_P, \tau_P, r_P) \in \mathcal{R}} \tau_P$, ties broken by smallest r_P $\triangleright$ fastest accepted plan
- 13: **return** $P^*, \mathcal{F}$

■ 4.4 Multi-Trajectory Batch Parallelism

A fused scan for one trajectory occupies one CUDA block, leaving most SMs without useful work. Simulation campaigns, however, naturally contain independent trajectories generated from different noise seeds, obstacle configurations, or filter parameters. Because these trajectories share the same computation but no recurrent state, they provide an outer parallel dimension without altering the sequential scan. For batch size B , we launch a one-dimensional grid of B Triton programs and assign trajectory b to program identifier b . Suppressing the innermost component offset, each input and output address has the form

$$\text{addr}(b, k) = \text{base} + b s_B + k s_N, \quad (8)$$

where s_B and s_N are the trajectory and time strides. Each program initializes its SINS and EKF state independently, so trajectories require neither synchronization nor communication.

This mapping yields a resource-based scaling model. Let S be the number of SMs, R_{SM} the register capacity of one SM, and R_{blk} the register allocation of one trajectory block. When registers are the active residency constraint, the number of resident blocks per SM and the corresponding device-wide saturation batch are approximated by

$$C_{\text{reg}} = \left\lfloor \frac{R_{\text{SM}}}{R_{\text{blk}}} \right\rfloor, \quad B_{\text{sat}} \approx S C_{\text{reg}}. \quad (9)$$

This approximation omits architectural limits on blocks, warps, shared memory, and scheduling, and therefore serves as a scaling guide rather than an exact occupancy model. For $B \leq S$, trajectories can be distributed across distinct SMs and aggregate throughput should grow nearly linearly. For $S < B \leq B_{\text{sat}}$, additional resident blocks increase contention for registers and execution units. Beyond B_{sat} , blocks execute in waves, so increasing B improves total work capacity but not instantaneous throughput. Section 5.5 tests these regimes with a batch-size sweep.

■ 4.5 Implementation Details

The complete scan is implemented as one `@triton.jit` kernel with launch grid $(B,)$. Each Triton program owns one trajectory, iterates over its N samples, and accesses fixed-shape state and matrix tiles through masked loads and stores. Precision types, dot-product mode, tile shape, and launch configuration are compile-time meta-parameters; specialization therefore introduces no runtime branch into the recurrent loop. Input and output buffers are allocated before launch, and compilation and autotuning are excluded from measured execution time [10].

On the evaluated Hopper GPU, Triton lowers eligible `tl.dot` operations to Tensor Core instructions for TF32 plans and uses CUDA-core arithmetic for IEEE plans. Scalar transcendental functions follow the backend's device-library lowering, while the analytical innovation inverse remains inside the program. We use conventional coalesced loads and stores for the per-step sensor stream because the scan consumes only a small sample at each iteration; the large recurrent state remains register-resident rather than being staged repeatedly through shared memory or TMA. This implementation matches the memory and execution mapping in Figure 5(b) while keeping the precision and launch choices explicit [11].

■ 5 Evaluation

■ 5.1 Experimental Setup

Experiments use an NVIDIA Hopper GPU with 78 SMs, PyTorch 2.11.0, CUDA 12.9, and Triton 3.6.0. The principal dataset contains 166,667 samples. The CPU eager fp64 implementation is used as the numerical golden reference. Reported GPU timings exclude compilation and use warm-up runs with the same problem shape as the measured execution. Throughput is reported in processed filter steps per second and, for batch experiments, trajectories per second.

■ 5.2 End-to-End Performance

This subsection will separate latency improvements for one trajectory from aggregate throughput improvements for batches. It will also reconcile performance values from different experiment rounds using the archived JSON results as the final source of truth.

■ 5.3 Effect of Kernel Fusion

This subsection will compare eager execution, compile-friendly restructuring, CUDA Graph replay, and the Triton mega-kernel. Profiler evidence will include kernel counts, launch intervals, time spent in solver calls, global-memory traffic, register use, and achieved SM throughput.

■ 5.4 Accuracy and Precision Trade-offs

The precision study will report throughput together with final-state deviation and X/Y/Z RMSE. The main comparison includes fp64, fp32, TensorFloat-32 matrix products, fp16 sensor input, and bfloat16 variants. Negative results are retained because they identify when reduced precision is unsafe in a long recursive filter.

■ 5.5 Batch Scalability and Resource Saturation

Batch sizes from one trajectory to beyond the register-limited residency point will be evaluated. The analysis will distinguish the linear region up to 78 blocks, the resource-contention region, and the saturation region around 312 blocks. Nsight Compute metrics will be used to connect the observed boundaries to register pressure and active blocks per SM.

■ 5.6 Ablation and Negative Results

The ablation study will isolate fusion, precision, and batch parallelism. Additional experiments will document the limited value of TensorFloat-32, CUDA streams for uniform trajectories, Tensor Memory Accelerator usage, warp specialization, and conventional tile or layout auto-tuning. These results support the claim that the optimization space of latency-bound micro-matrix recursion differs from that of large dense linear algebra.

■ 6 Related Work

■ 6.1 Downhole Tractors and In-Pipe Robots

Recent field studies demonstrate that downhole tractors can convey wireline tools and support intervention, logging, and perforation in highly deviated and horizontal wells [8, 9]. Existing work primarily addresses traction capability, tool conveyance, and operational reliability. Our work complements it by studying the computational behavior of long-horizon multi-sensor positioning simulation and by optimizing its recursive SINS/EKF workload for GPU execution.

■ 6.2 GPU-Accelerated Kalman Filtering

GPU implementations of Kalman filtering expose parallelism at several different levels. Early work mapped the matrix operations within a filter step to CUDA and reported speedups when the matrices were large enough to amortize data movement and kernel-launch costs [12]. Application-specific systems more often exploit independent estimation problems. The GPU track-fitting implementation of Ai et al. [13] processes many particle tracks concurrently, while FastTrack parallelizes the prediction and update of multiple tracked objects and integrates the

Table 5 End-to-end performance summary. The final manuscript will report means and variability over repeated runs.

Version	Method	Steps/s	Speedup
v0	CPU eager fp64	1,866	1.0×
v2	CUDA Graph fp64	2,640	1.41×
v3	Triton fused fp64	16,491	8.8×
v4	Triton fused fp32	242,737	130×
v5	fp32, batch 78	17.8–19.1M	approx. 10,000×
v7	fp32, batch 312	46.3M	approx. 24,800×

filter with the surrounding tracking pipeline [14]. These designs are effective when a workload supplies many simultaneous filters, although they do not remove the recurrence or fine-grained execution overhead of one long, small-state trajectory.

Another line of work derives temporally parallel filtering and smoothing algorithms. Särkkä and García-Fernández formulate Bayesian smoothing operations as associative elements, enabling prefix-scan algorithms with logarithmic span [15]. Such reformulations are attractive when the model admits associative composition and parallel latency is the primary goal. The downhole traction robot positioning simulation studied here retains the original nonlinear SINS propagation, measurement selection, error feedback, and covariance recursion in their program order. We shorten this sequential path by fusing the complete scan and keeping its recurrent state local, then use independent simulation trajectories—rather than time steps from one trajectory—to occupy the GPU.

■ 6.3 MegaKernel and Kernel Fusion

Kernel fusion combines operations so that intermediate values can remain on chip and repeated kernel launches can be avoided. MegaKernels enlarge this scope to a substantial computation region; for example, Jia et al. [16] place the stages of fast convolution in one GPU kernel to reuse intermediate data and reduce launch and synchronization overhead. Recent compiler and systems research has extended fusion beyond manually selected, memory-bound operator chains. Welder constructs a tile graph to jointly search fusion and tile schedules according to memory traffic [17]. Chimera uses an analytical model to select fusion plans for compute-intensive operators [18], and MCFuser combines memory-bound operators with compute-intensive operators while controlling recomputation and synchronization costs [19]. These systems show that profitable fusion depends on both the operator graph and the hardware cost created by a candidate fused region.

More recent systems search larger transformation spaces. Mirage represents tensor programs at kernel, thread-block, and thread levels, allowing its superoptimizer to combine algebraic transformations, schedules, and custom-kernel generation [20]. STOF uses compilation templates and a two-stage search to select adaptive fusion strategies for sparse Transformer inference [21]. These approaches primarily optimize tensor graphs whose operators retain substantial spatial parallelism. Our target presents a different fusion boundary: a complete sequential SINS/EKF trajectory is compiled as one Triton program [10], and its small navigation state and covariance remain program-local across time steps. Parallelism is recovered across independent trajectories, while fusion reduces the latency of each recurrent scan. This specialization complements graph-level fusion systems by addressing a long-lived kernel with an unavoidable loop-carried dependency.

■ 6.4 Mixed Precision for Scientific Computing

Mixed-precision methods have been widely studied in scientific computing and GPU acceleration [22]. Baboulin et al. [23] combine lower-precision kernels with higher-precision residual evaluation and correction in numerical linear algebra. Haidar et al. [24] further map

low-precision factorization and solves to GPU Tensor Cores while retaining higher precision for accuracy-critical operations.

Automatic precision selection provides a complementary approach. Precimonious searches variable- and operation-level assignments under an application-provided error constraint [25]. CoMP performs operator-wise precision selection for neural-network training through convergence-aware and performance-oriented sampling [26]. Its search also considers format-conversion overhead and the availability of Tensor Core execution, illustrating the importance of evaluating precision plans against both numerical requirements and the characteristics of the target GPU.

■ 7 Conclusion

This paper presents TRIDENT, a three-axis approach to GPU acceleration of SINS/EKF fusion positioning for downhole traction robots. The central observation is that the workload is limited by launch latency, sequential dependency, and idle SMs rather than by arithmetic throughput. Whole-scan kernel fusion removes fragmented execution, component-aware precision matches numerical requirements to GPU hardware, and multi-trajectory mapping exposes the independent work needed to fill the device. The resulting implementation changes a naive GPU slowdown into substantial single-trajectory acceleration and scalable batch throughput while preserving positioning accuracy. Future work will evaluate TRIDENT across additional state-estimation models, trajectory distributions, and GPU architectures, and will investigate structure-aware reductions in covariance computation.

■ Acknowledgements

To be completed.

■ Competing interests

The authors declare that they have no competing interests or financial conflicts to disclose.

■ References

- [1] Liu Y, Chen W, Bai Y, Liang X, Li G, Gao W, Lin L. Aligning cyber space with physical world: A comprehensive survey on embodied AI. *IEEE/ASME Transactions on Mechatronics*, 2025, 30(6): 7253–7274
- [2] He Z, Teng S, Lin T Y, Ghaffari M, Gu Y. Legged robot state estimation within non-inertial environments. In: 2024 IEEE 63rd Conference on Decision and Control (CDC). 2024, 2469–2474
- [3] Ou G, Li D, Li H. Leg-KILO: Robust kinematic-inertial-lidar odometry for dynamic legged robots. *IEEE Robotics and Automation Letters*, 2024, 9(10): 8194–8201
- [4] Duan J, Yu S, Tan H L, Zhu H, Tan C. A survey of embodied AI: From simulators to research tasks. *IEEE Transactions on Emerging Topics in Computational Intelligence*, 2022, 6(2): 230–244
- [5] Bloesch M, Hutter M, Hoepfner M, Leutenegger S, Gehring C, Remy C D, Siegwart R. State estimation for legged robots: Consistent fusion of leg kinematics and IMU. In: *Robotics: Science and Systems VIII*. 2012
- [6] Tranzatto M, Miki T, Dharmadhikari M, Bernreiter L, Kulkarni M, Mascari F, Andersson O, Khattak S, Hutter M, Siegwart R, Alexis K. CERBERUS in the DARPA subterranean challenge. *Science Robotics*, 2022, 7(66): eabp9742

- [7] Makoviychuk V, Wawrzyniak L, Guo Y, Lu M, Storey K, Macklin M, Hoeller D, Rudin N, Allshire A, Handa A, State G. Isaac Gym: High performance GPU-based physics simulation for robot learning. arXiv preprint arXiv:2108.10470, 2021
- [8] He L, McAllister J, Hawkins J, Turner M. Application of downhole tractor in gas well zonal isolation. In: SPE/ICoTA Well Intervention Conference and Exhibition. 2020
- [9] Chawla T S. Downhole wireline tractor glide in horizontal wells of UAE. In: Mediterranean Offshore Conference. 2024
- [10] Tillett P, Kung H T, Cox D. Triton: An intermediate language and compiler for tiled neural network computations. In: Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages. 2019, 10–19
- [11] NVIDIA Corporation. NVIDIA H100 Tensor Core GPU Architecture. Architecture Whitepaper, 2022
- [12] Huang M Y, Wei S C, Huang B, Chang Y L. Accelerating the Kalman filter on a GPU. In: Proceedings of the 17th IEEE International Conference on Parallel and Distributed Systems. 2011
- [13] Ai X, Mania G, Gray H M, Kühn M, Styles N. A GPU-based Kalman filter for track fitting. Computing and Software for Big Science, 2021, 5(1): 20
- [14] Liu C, Li H, Wang Z. FastTrack: A highly efficient and generic GPU-based multi-object tracking method with parallel Kalman filter. International Journal of Computer Vision, 2024, 132(5): 1463–1483
- [15] Särkkä S, García-Fernández Á F. Temporal parallelization of bayesian smoothers. IEEE Transactions on Automatic Control, 2021, 66(1): 299–306
- [16] Jia L, Liang Y, Li X, Lu L, Yan S. Enabling efficient fast convolution algorithms on GPUs via MegaKernels. IEEE Transactions on Computers, 2020
- [17] Shi Y, Yang Z, Xue J, Ma L, Xia Y, Miao Z, Guo Y, Yang F, Zhou L. Welder: Scheduling deep learning memory access via tile-graph. In: Proceedings of the 17th USENIX Symposium on Operating Systems Design and Implementation. 2023, 701–718
- [18] Zheng S, Chen S, Song P, Chen R, Li X, Yan S, Lin D, Leng J, Liang Y. Chimera: An analytical optimizing framework for effective compute-intensive operators fusion. In: Proceedings of the 29th IEEE International Symposium on High-Performance Computer Architecture. 2023, 1113–1126
- [19] Zhang Z, Yang D, Zhou X, Cheng D. MCFuser: High-performance and rapid fusion of memory-bound compute-intensive operators. In: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis. 2024, 1–15
- [20] Wu M, Cheng X, Liu S, Shi C, Ji J, Ao M K, Velliengiri P, Miao X, Padon O, Jia Z. Mirage: A multi-level superoptimizer for tensor programs. In: Proceedings of the 19th USENIX Symposium on Operating Systems Design and Implementation. 2025, 21–38
- [21] Dai W, Deng H, Rong M, Yang X, Liu H, Liu F, Yang H, Cao Q, Sun Q. Accelerating sparse transformer inference on GPU. In: Proceedings of the 31st ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming. 2026, 620–634
- [22] Abdelfattah A, Anzt H, Boman E G, Carson E, Cojean T, Dongarra J, Fox A, Gates M, Higham N J, Li X S, Loe J, Luszczek P, Pranesh S, Rajamanickam S, Ribizel T, Smith B F, Swirydowicz K, Thomas S, Tomov S, Tsai Y M, Yang U M. A survey of numerical linear algebra methods utilizing mixed-precision arithmetic. The International Journal of High Performance Computing Applications, 2021, 35(4): 344–369
- [23] Baboulin M, Buttari A, Dongarra J, Kurzak J, Langou J, Langou J, Luszczek P, Tomov S. Accelerating scientific computations with mixed precision algorithms. Computer Physics Communications, 2009, 180(12): 2526–2533
- [24] Haidar A, Tomov S, Dongarra J, Higham N J. Harnessing GPU tensor cores for fast FP16 arithmetic to speed up mixed-precision iterative refinement solvers. In: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis. 2018, 603–613
- [25] Rubio-González C, Nguyen C, Nguyen H D, Demmel J, Kahan W, Sen K, Bailey D H, Iancu C, Hough D. Precimonious: Tuning assistant for floating-point precision. In: Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis. 2013, 1–12
- [26] Dai W, Jia Z, Bai Y, Sun Q. Convergence-aware operator-wise mixed-precision training. CCF Transactions on High Performance Computing, 2025, 7: 43–57